§ 11. Прикладные структуры и выбор решения

В предыдущих главах мы выбирали представление под конкретные операции: сумму на отрезке, переход между версиями или извлечение минимума. Теперь добавим ограничения среды. Данные могут не помещаться в оперативную память; запрос может описывать область карты; граф может терять рёбра; важна бывает не средняя скорость, а задержка одной операции. Эти условия меняют выбор структуры даже при одинаковом числе элементов.

Подробный пример главы — skip list, или список с пропусками. Мы выведем его инварианты и ожидаемую сложность, реализуем изменяемое упорядоченное множество на C++17. Затем сравним его с 2–3-деревом, LSM-tree и пространственными индексами, обозначим границы динамической связности и очередей Бродала–Окасаки. Обзорные разделы объясняют механизмы и ограничения; они не заменяют полные реализации всех перечисленных структур.

11.1. Что измерять до выбора структуры

Фраза «нужно быстро хранить миллион записей» не задаёт задачу. Какие запросы нужны: найти точный ключ, ближайший справа, все ключи диапазона, геометрическое пересечение? Какие изменения разрешены? Должны ли сохраняться старые состояния? Где находятся данные и какую стоимость мы считаем?

В модели RAM считаются обращения к словам памяти и сравнения. При работе с диском удобнее считать передачи блоков: чтение одного блока с B записями гораздо дешевле B независимых обращений. Указатели, хорошие в RAM, могут приводить к случайным чтениям на диске. В многопоточной среде добавляются синхронизация и правила времени жизни узлов; последовательный алгоритм не становится потокобезопасным от одной замены указателя на атомарный.

Различайте три гарантии из § 1. Худший случай ограничивает любую отдельную операцию. Амортизированная оценка относится ко всей последовательности и допускает дорогой отдельный шаг. Ожидаемая оценка усредняет стоимость по случайному выбору структуры; она не обещает одинаковой задержки при каждом запуске. В skip list вероятность относится к высотам узлов, а не к предполагаемой «случайности» ключей пользователя.

Редакторский сценарий. Сервис хранит числовые идентификаторы и поддерживает вставку, удаление и поиск первого идентификатора не меньше заданного. Все данные помещаются в RAM, исторические версии не нужны. Skip list и сбалансированное дерево подходят по интерфейсу. Если вместо этого нужны прямоугольники на карте, числового порядка одного ключа недостаточно. Если основная работа — поток записей на диск, стоит рассмотреть организацию с отложенными слияниями.

Требование	Что проверить в первую очередь
Строгая задержка операции	Худший случай, выделение и освобождение памяти
Частые записи на диск	Последовательность записи, цена слияний
Много диапазонных чтений	Порядок ключей, число просматриваемых блоков
Поиск по геометрии	Размерность, перекрытие областей, точный фильтр
Удаление рёбер	Лес или общий граф, доступность всей истории
Исторические запросы	Персистентность, снимки или офлайн-откат

(c) Aleksey Tolstikov

11.2. Skip list: упорядоченные списки разных плотностей

Нижний уровень содержит все ключи в строго возрастающем порядке. Каждый следующий уровень — подпоследовательность предыдущего. Узел высоты h имеет ссылки уровней $0, \dots, h-1$. Служебная голова присутствует на всех уровнях; её поле ключа не участвует в сравнении, поэтому не нужно резервировать минимальное целое значение.

При вставке начинаем с высоты 1 и повышаем её, пока независимая честная монета даёт продолжение. Получаем $P(h \geq k) = 2^{-(k-1)}$. Большинство узлов низкие, высокие дают длинные переходы. Верхние уровни не делят пространство на фиксированные блоки и не требуют вращений.

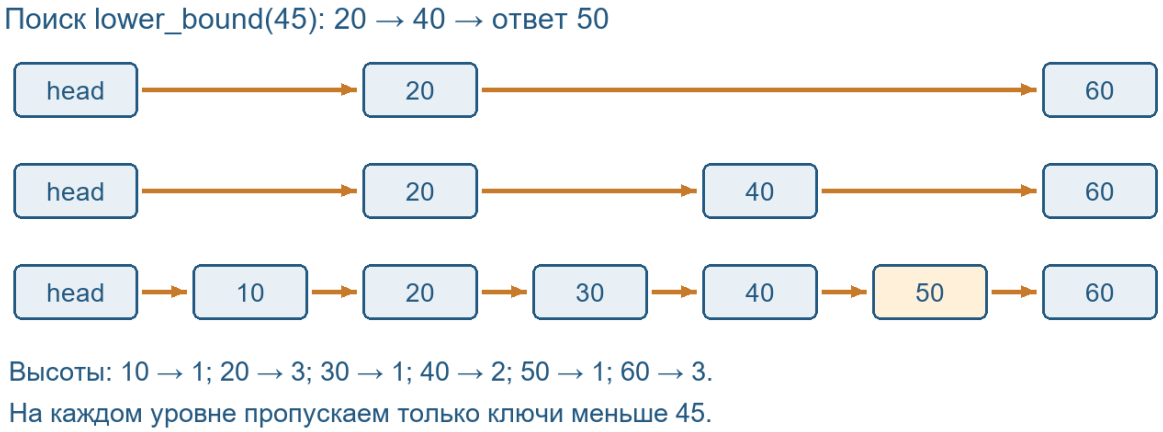


Рисунок 1: Нижний уровень содержит все ключи; верхние позволяют пропускать целые участки. Высоты на схеме выбраны вручную.

Для `lower_bound(x)` начинаем у головы на самом высоком непустом уровне. Пока следующий ключ строго меньше x, идём вправо. Если дальше идти нельзя, спускаемся на уровень ниже, оставаясь в том же узле. После уровня 0 следующий узел — первый с ключом не меньше x, либо ответа нет.

На схеме ключи $10, 20, 30, 40, 50, 60$ имеют высоты $1, 3, 1, 2, 1, 3$. Для $x = 45$ верхний уровень позволяет перейти к 20, но не к 60. На среднем уровне переходим к 40. На нижнем следующий ключ 50, поэтому ответ 50. Для $x = 40$ переход к самому 40 запрещён условием «строго меньше», и он станет ответом, а не будет пропущен.

Инвариант поиска. Текущий узел расположен до искомой позиции; пропущенные ключи меньше x. На текущем уровне после остановки следующая ссылка ведёт к ключу не меньше x либо в конец. Спуск не теряет кандидатов: более низкий уровень лишь добавляет промежуточные узлы между текущим узлом и следующим. На уровне 0 промежуточных скрытых ключей уже нет.

Для вставки запоминаем `before[i]` — последнего предшественника ключа на каждом просмотренном уровне. Новый узел встраивается между ним и прежним следующим узлом на всех уровнях своей высоты. Если ключ уже существует, ничего не меняем. Удаление делает обратное перенаправление всех входящих ссылок на удаляемый узел.

11.3. Почему стоимость ожидаемо логарифмическая

Ожидаемая высота одного узла равна сумме вероятностей достижения уровней:

$$E[h] = \sum_{k \geq 1} P(h \geq k) = 1 + \frac{1}{2} + \frac{1}{4} + \cdots = 2.$$

Значит, ожидаемое число ссылок у n узлов равно $2n$. Эта оценка не требует независимости между слагаемыми, достаточно линейности ожидания. Независимость бросков нужна для распределения высот и анализа маршрута поиска.

Вероятность существования узла высоты хотя бы k не превосходит $n/2^{k-1}$ по объединению событий. Поэтому выше примерно $\log_2 n$ уровней остаётся убывающий геометрический хвост. Ожидаемая максимальная высота — $O(\log(n+1))$.

Чтобы оценить горизонтальные переходы, мысленно проследим найденный путь назад. На данном уровне идём влево до узла, который поднят на следующий уровень. Каждый подходящий узел поднимается с вероятностью $1/2$, поэтому ожидаемое число просмотренных узлов до подъёма ограничено константой. На $O(\log(n+1))$ существенных уровнях получается ожидаемо логарифмический путь. Корректность поиска детерминирована: случайность влияет на длину пути, а не на ответ.

У идеальной модели нет предела высоты. В реализации ниже предел $H = 32$. При обрезании распределения сверху список всё равно корректен, но слишком низкий предел ухудшает время: на верхнем уровне могут остаться $\Theta(n/2^{H-1})$ горизонтальных переходов. Для сохранения стандартной оценки выбираем $H \geq \lceil \log_2(n+1) \rceil$. Здесь предполагается не более 2^{31} одновременно хранимых ключей; реальный лимит памяти обычно существенно меньше.

Итог для поиска, вставки и удаления — ожидаемо $O(\log(n+1))$ времени при указанных предпосылках. В худшем случае все узлы имеют высоту 1 и поиск просматривает весь список. Общая оценка худшего случая — $O(n+H)$ на операцию, ожидаемой памяти — $O(n+H)$, худшей памяти — $O(nH)$ ссылок. Для фиксированного H последняя оценка линейна по n, но с большой константой.

Псевдослучайный генератор в программе позволяет воспроизводить тесты. Теоретический анализ предполагает независимые случайные высоты и историю запросов, не выбираемую противником по скрытым броскам. Фиксированный seed не даёт криптографической защиты и не превращает ожидаемую гарантию в гарантию худшего случая.

11.4. Реализация множества на C++17

Контракт класса: уникальные ключи типа `int`; `insert` и `erase` возвращают, изменилось ли множество; `contains` отвечает о наличии; `lower_bound` возвращает `optional<int>`; пустой ответ обозначается `nullopt`. Допустимы оба края диапазона `int`, без специальных значений для начала и конца.

Полный файл — `code/11-skip-list.hpp`. Требуются `<array>`, `<cstddef>`, `<cstdint>`, `<optional>`, `<random>` и `<vector>`. Нижние листинги — последовательные части одного класса.

(c) Aleksey Tolstikov

```
class SkipSet {
    static constexpr int MAX_LEVEL=32;
    struct Node {
        int key;
        std::vector<Node*> next;
        Node(int x,int height):key(x),next(height,nullptr) {}
    };
    Node head{0,MAX_LEVEL};
    int levels=1;
    std::size_t count=0;
    std::mt19937_64 random;

    int random_height() {
        int h=1;
        while (h<MAX_LEVEL && (random()&1ULL)) ++h;
        return h;
    }
    Node* predecessors(int x,std::array<Node*,MAX_LEVEL>& before) {
        Node* p=&head;
        for (int i=levels-1;i>=0;--i) {
            while (p->next[i] && p->next[i]->key<x)
                p=p->next[i];
            before[i]=p;
        }
        return p->next[0];
    }
```

Метод `predecessors` возвращает кандидата на уровне 0 и заполняет предшественников активных уровней. Когда текущий узел достигнут по ссылке уровня i, его высота больше i, поэтому обращение к `next[i]` допустимо. После спуска это свойство сохраняется.

Класс владеет узлами. Копирование запрещено, чтобы два объекта не освободили одни и те же адреса. Деструктор обходит только нижний уровень: каждый узел освобождается ровно один раз, верхние ссылки не владеют памятью.

```
public:
    explicit SkipSet(std::uint64_t seed=20261031):random(seed) {}
    SkipSet(const SkipSet&)=delete;
    SkipSet& operator=(const SkipSet&)=delete;
    ~SkipSet() {
        Node* p=head.next[0];
        while (p) {
            Node* next=p->next[0];
            delete p;
            p=next;
        }
    }
    std::size_t size() const { return count; }
```

Чтение не меняет список и не выделяет узлы. Указатель проходит те же уровни, что и при подготовке вставки, но массив предшественников ему не нужен.

```
    std::optional<int> lower_bound(int x) const {
        const Node* p=&head;
        for (int i=levels-1;i>=0;--i)
            while (p->next[i] && p->next[i]->key<x)
                p=p->next[i];
        p=p->next[0];
        if (!p) return std::nullopt;
        return p->key;
    }
    bool contains(int x) const {
        auto answer=lower_bound(x);
        return answer && *answer==x;
    }
```

`size` читает сохранённый счётчик за $O(1)$. `contains` использует тот же `lower_bound`, не создавая второй алгоритм поиска с иными правилами сравнения. Деструктор требует $O(n)$ времени; это отдельная операция освобождения всей структуры, а не часть оценки поиска.

```
bool insert(int x) {
        std::array<Node*,MAX_LEVEL> before{};
        Node* candidate=predecessors(x,before);
        if (candidate && candidate->key==x) return false;
        int h=random_height();
        Node* fresh=new Node(x,h);
        for (int i=levels;i<h;++i) before[i]=&head;
        for (int i=0;i<h;++i) {
            fresh->next[i]=before[i]->next[i];
            before[i]->next[i]=fresh;
        }
        if (h>levels) levels=h;
        ++count;
        return true;
    }
```

Если новый узел выше всех прежних, предшественником на новых уровнях служит голова. Память под узел выделяется до изменения ссылок. При неудаче выделения логическое множество остаётся прежним. После успешного создания выполняются только записи указателей и счётчиков.

```
bool erase(int x) {
        std::array<Node*,MAX_LEVEL> before{};
        Node* candidate=predecessors(x,before);
        if (!candidate || candidate->key!=x) return false;
        int h=static_cast<int>(candidate->next.size());
        for (int i=0;i<h;++i)
            before[i]->next[i]=candidate->next[i];
        delete candidate;
        --count;
        while (levels>1 && !head.next[levels-1]) --levels;
        return true;
    }
```

После удаления высокой башни верхние уровни могут опустеть: уменьшаем `levels`, оставляя минимум один уровень для головы. Узел сначала исключается из всех списков и только затем освобождается. Публичный интерфейс не выдаёт внешних указателей на узлы, поэтому клиенту не приходится хранить адреса после удаления.

В полном заголовке также есть диагностические `values()` и `valid()`. Первый проходит уровень 0, второй проверяет, что каждый верхний список совпадает с подпоследовательностью узлов достаточной высоты.

Они не вызываются внутри быстрых операций. Эта реализация последовательная и изменяемая; она не предоставляет персистентность или конкурентное изменение.

11.5. Ручная трасса и независимая проверка

Вернёмся к схеме из 11.2. Вставим 35 и для ручного разбора назначим ему высоту 2. На уровне 2 предшественник — 20, на уровне 1 — тоже 20, на уровне 0 — 30. Новые ссылки получаются только на уровнях 0 и 1: $30 \to 35 \to 40$ и $20 \to 35 \to 40$. Верхний переход $20 \to 60$ не меняется. В реальной программе высоту выбирает генератор; схема не описывает конкретный seed.

Теперь удалим 20. До удаления его предшественник на верхних уровнях — голова, на нижнем — 10. На уровне 2 голова получает ссылку на 60, на уровне 1 — на 35, на уровне 0 узел 10 ссылается на 30. Остальные связи не меняются. Обход нижнего уровня выдаёт $10, 30, 35, 40, 50, 60$.

Операция после этих изменений	Результат	Почему
`contains(20)`	false	Ответ `lower_bound(20)` равен 30
`lower_bound(35)`	35	Равный ключ не пропускается
`lower_bound(36)`	40	35 меньше запроса
`lower_bound(61)`	нет ответа	Все ключи меньше 61
`insert(35)`	false	Множество не хранит кратности
`erase(20)`	false	Повторное удаление ничего не меняет

Эталон — `std::set<int>`. Сравниваем результат каждой операции, размер и периодически весь отсортированный набор. Проверяем пустое множество, повторные вставки и удаления, ключи `INT_MIN` и `INT_MAX`, возрастающие и убывающие последовательности. Несколько seed дают разные формы при одинаковом контракте.

Проверка `valid` должна смотреть не только на сортировку уровня 0. Потерянная ссылка наверху может долго не портить ответ, а лишь замедлять поиск. Проверяем точное совпадение каждого верхнего списка с нужной подпоследовательностью нижнего, число узлов и максимальную активную высоту. Эксперимент не доказывает ожидаемую оценку: её основание — вероятностный анализ из 11.3.

Идея skip list принадлежит William Pugh; его технический отчёт содержит дальнейшие операции, включая разделение и слияние. Реализация, трасса и проверочный драйвер здесь разработаны для учебника.

11.6. 2–3-дерево: баланс без случайности

В 2–3-дереве узел хранит один или два упорядоченных ключа. Внутренний узел с одним ключом имеет два ребёнка, с двумя — три; листья не имеют детей. Все листья находятся на одной глубине. Между соседними ключами родителя лежит соответствующий диапазон ключей ребёнка. Рассматриваем множество без повторов.

Если высота равна h рёбрам, минимальное число ключей — $2^{h+1} - 1$, максимальное — $3^{h+1} - 1$. Эти границы получаются из полностью двухветвящегося и полностью трёхветвящегося дерева. Следовательно, $h = \Theta(\log(n + 1))$. Поиск просматривает до двух ключей на каждом уровне, то есть логарифмичен в худшем случае.

При вставке спускаемся к листу. Если в нём один ключ, добавляем второй. Если временно получилось три ключа $a < b < c$, разбиваем узел: b поднимается в родителя, a и c остаются в двух узлах.

Переполнение родителя обрабатывается так же. Если расщепился корень, создаётся новый корень; все листья одновременно оказываются на уровень глубже.

Редакторский пример. После вставок 10,20,30 корень содержит 20, листья — 10 и 30. Затем 40 помещается в правый лист. Вставка 50 поднимает 40: корень становится $[20, 40]$, листья — $[10]$, $[30]$, $[50]$. После вставок 60 и 70 переполнится правый лист, затем корень. Новый корень 40 имеет детей 20 и 60, под ними листья 10,30,50,70.

При удалении может появиться узел без ключей. Если соседний брат имеет два ключа, перераспределяем ключ через родителя, сохраняя порядок. Иначе сливаем пустой узел, ключ-разделитель родителя и брата. Дефицит может подняться на следующий уровень. Пустой корень заменяется единственным ребёнком. В обоих случаях сохраняется одинаковая глубина листьев; обработка идёт вдоль одного пути.

Это гарантированная балансировка, в отличие от случайных высот skip list. 2–3-дерево также помогает понять B-деревья с большим числом ключей в узле. Во внешней памяти узел подбирают под блок: большое ветвление уменьшает число чтений. Само 2–3-дерево с двумя-тремя детьми ещё не обеспечивает эффективного заполнения дискового блока.

11.7. LSM-tree: перенос работы с записи на слияние

LSM-tree организует словарь как несколько упорядоченных компонентов. Новые записи сначала попадают в буфер в RAM; заполненный буфер превращается в отсортированный неизменяемый файл. Фоновые слияния объединяют файлы и убирают устаревшие записи. Идея — собирать много мелких изменений в крупную последовательную работу.

Запись должна хранить ключ, значение и порядок версий. Удаление представляется специальной записью — tombstone. При чтении одного ключа среди видимых кандидатов выбирается самый новый; метка удаления означает отсутствие значения. Нельзя просто удалить ключ из буфера: более старая запись на диске иначе снова станет видимой.

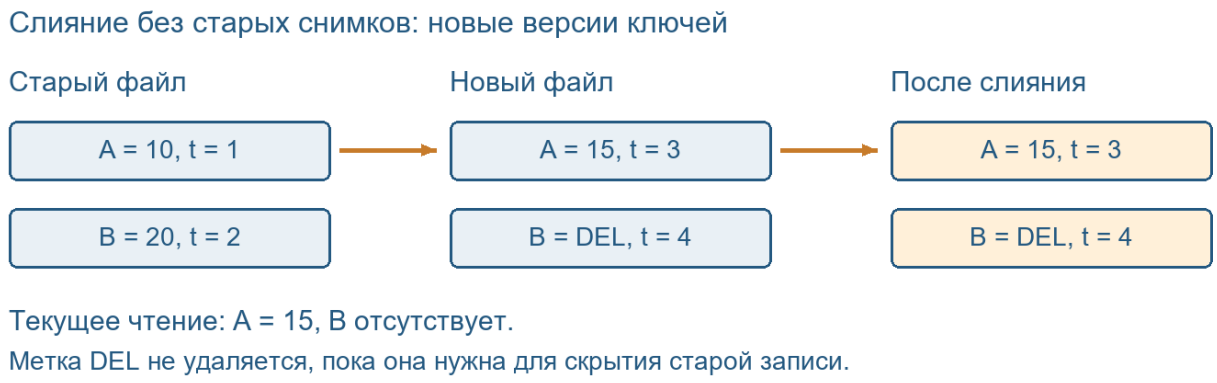


Рисунок 2: Слияние для текущего состояния, без старых снимков: выбирается новая запись ключа, включая метку удаления.

В редакторском примере без запросов к старым снимкам старый файл содержит `(A,10,t=1)` и `(B,20,t=2)`, новый — `(A,15,t=3)` и `(B,DEL,t=4)`. Текущее чтение A возвращает 15, B отсутствует. Снимок на момент 2 должен видеть A=10 и B=20, если такой контракт поддерживается. Удалить старые записи и tombstone безопасно только тогда, когда они больше не нужны ни оставшимся компонентам, ни разрешённым снимкам.

Для анализа рассмотрим **учебную модель**, а не готовую СУБД: по одному отсортированному массиву каждого размера $1, 2, 4, \ldots$; вставка одноэлементного массива сливает равные размеры, как перенос в двоичном счётчике. Для N уникальных вставок каждый элемент участвует не более чем в $\lceil \log_2 N \rceil$ слияниях. Получаем $O(N \log(N+1))$ суммарной работы и амортизированное $O(\log(N+1))$ на вставку. Отдельная вставка при заполненных младших уровнях может переписать $\Theta(N)$ элементов.

Поиск в этой модели проверяет до $O(\log N)$ массивов. Обычный двоичный поиск в каждом даёт суммарно $O(\log^2(N+1))$ сравнений в худшем случае. Получить более быстрый запрос можно дополнительными индексами и фильтрами, но их стоимость надо учитывать отдельно. Эта оценка не является универсальной формулой для всех LSM-движков.

В реальных системах сравнивают усиление записи, чтения и объёма хранения: сколько физических данных записывается, сколько блоков читается и сколько места занято относительно полезных данных. Политики compaction меняют баланс этих затрат. Например, документация RocksDB описывает уровни с непересекающимися диапазонами файлов внутри уровня, кроме L0, где свежие файлы могут перекрываться. Это конкретная организация, а не обязательное свойство всякого LSM-tree.

Фильтр Блума помогает отбрасывать файлы, в которых точно нет ключа, но положительный ответ фильтра требует проверки. Он не заменяет поиск и не гарантирует ускорения произвольного диапазонного сканирования. При сканировании нужно объединить отсортированные потоки, учесть версии и удаления.

Сохранность после сбоя — отдельный контракт. Для подтверждения записи может требоваться журнал и его синхронизация с носителем; один буфер в RAM этого не даёт. Название «persistent storage» означает долговременное хранение, а персистентность §7–10 — доступность старых логических версий. Это связанные, но не тождественные свойства.

11.8. Пространственные индексы: R, R+ и R*

Числовое дерево поиска выбирает одну ветвь по сравнению с ключом. В геометрии объект имеет несколько координат и протяжённость. R-дерево группирует объекты в страницы; каждая запись внутреннего узла хранит ограничивающий прямоугольник дочерней группы. Лист хранит прямоугольник объекта и ссылку на сам объект. Дерево сбалансировано по глубине листьев, но прямоугольники групп могут пересекаться.

При запросе прямоугольной области идём во все дочерние группы, чьи прямоугольники пересекают запрос. На листе проверяем кандидатов. Если реальный объект не прямоугольный, пересечение его оболочки с запросом — только необходимое условие: нужен точный геометрический тест.

Редакторский пример. Две группы имеют оболочки $[0, 6] \times [0, 4]$ и $[4, 9] \times [2, 7]$. Запрос $[5, 5.5] \times [3, 3.5]$ пересекает обе. Даже очень узкий запрос может потребовать две ветви; при плохом перекрытии — много ветвей на каждом уровне. Логарифмическая высота не доказывает время запроса $O(\log n + k)$, где k — число ответов: стоимость зависит и от ложных кандидатов.

R+-дерево устраняет перекрытие внутренних областей разбиения; цена — возможное присутствие одного протяжённого объекта в нескольких частях индекса. При выдаче объектов следует учитывать повторы идентификаторов. Это отличается от простого запрета пересекаться самим объектам. Такой подход изложен в работе Sellis, Roussopoulos и Faloutsos.

R*-дерево допускает перекрытие, но улучшает выбор поддерева и разделение переполненных страниц, учитывая площадь, периметр оболочек и перекрытие. Часть элементов может повторно вставляться вместо немедленного локального разделения. Исходная статья обосновывает конструкцию экспериментами; это не обещание логарифмического худшего времени любого геометрического запроса.

Для измерения полезны число посещённых страниц, число кандидатов перед точным фильтром, число выданных объектов и стоимость обновлений. Нужно варьировать размеры и вытянутость объектов, плотность и размер запроса. В высокой размерности оболочки часто перекрываются сильнее, поэтому успех двумерного примера нельзя автоматически переносить на десятки координат.

11.9. Динамическая связность: почему удаления сложнее

Для неориентированного графа интерфейс может содержать `add_edge`, `remove_edge` и `connected(u,v)`. Если рёбра только добавляются, достаточно DSU: объединение по размеру и сжатие путей дают почти постоянную амортизированную стоимость $O(\alpha(n))$. Удаление произвольного ребра DSU не поддерживает.

Рассмотрим треугольник с рёбрами $(1,2),(2,3),(1,3)$. Объединения для первых двух рёбер создали одну компоненту; третье ребро не изменило DSU. После удаления $(2,3)$ граф остаётся связным через $(1,3)$. Простое «отменить соответствующее объединение» дало бы неверный ответ. Структура компонент потеряла информацию о запасных путях.

Для динамического **леса** циклов нет: удалённое ребро действительно разделяет дерево. Link/cut tree и другие динамические деревья позволяют менять лес и поддерживать сведения на путях. В общем графе дополнительно надо искать заменяющее ребро между двумя частями после удаления ребра остовного леса. Одной структуры динамического дерева для этого недостаточно.

Если вся история известна заранее, применим знакомый по §10 откат. Каждому вхождению ребра сопоставляем полуинтервал его жизни $[add, remove)$. Покроем его $O(\log Q)$ узлами дерева отрезков по времени. Во время DFS добавляем рёбра текущего узла в DSU, отвечаем на запросы листа и при выходе откатываем состояние.

Для отката удобна DSU с объединением по размеру **без сжатия путей**. Высота ограничена $O(\log n)$: каждый раз, когда корень становится ребёнком, размер его новой компоненты как минимум удваивается. В журнал записываем старые значения родителя и размера при успешном объединении. Перед входом в узел сохраняем длину журнала; при выходе восстанавливаем её. Сжатие путей тоже можно журналировать, но тогда простой анализ и константное число записей на union больше не действуют.

Если есть K интервалов жизни и Q моментов времени, хранение интервалов в дереве требует $O(K \log Q + Q)$, обработка — $O(K \log Q \log n + Q \log n)$ времени при запросе связности в каждом листе. Добавляются $O(n)$ памяти DSU и журнал текущего пути. При повторных параллельных рёбрах нужно заранее определить, различаются ли их идентификаторы или учитывается счётчик кратности.

Это офлайн-решение: оно не отвечает на неизвестную будущую историю в режиме реального времени. Оно показывает, как дерево отрезков, инвариант DFS и откат соединяются в один алгоритм. Сложные онлайн-структуры полной динамической связности — отдельная тема углубления.

11.10. Куча Бродала–Окасаки и цена сильной гарантии

В §9 левосторонняя куча обеспечивала логарифмические вставку и слияние в худшем случае. Для функциональных очередей можно достичь более сильных границ: `findMin`, `insert`, `meld` за $O(1)$, `deleteMin` за $O(\log(n+1))$ в худшем случае. Это результат Brodal и Okasaki, а не свойство нашего кода левосторонней кучи.

Их построение связано с биномиальными очередями: устраняется каскад связей при вставке, минимум хранится в общем корне, а очередь может содержать другие очереди. Последний приём называют структурным самоприменением, или data-structural bootstrapping. Для получения заявленных оценок

нужны точные инварианты вложенной структуры; хранение списка отложенных слияний само по себе их не обеспечивает.

Почему не сделать все четыре операции константными? В модели сравнений вставим n различных ключей и затем последовательно извлечём минимум. Получится сортировка. Если и удаление занимало бы $O(1)$, весь процесс потребовал бы $O(n)$ сравнений, что противоречит нижней границе $\Omega(n \log n)$. Поэтому дорогая операция не исчезает: её цена сосредоточена в извлечении минимума.

Этот результат важен как ориентир достижимых гарантий. Для прикладного выбора дополнительно оценивают сложность реализации, константы, выделение памяти и набор действительно нужных операций. Нельзя обещать этим же интерфейсом произвольный `decrease_key`, удаление по внешнему указателю или конкурентный доступ без отдельного алгоритма и анализа.

11.11. Как завершить проект структуры данных

Выбор решения можно оформить коротким доказательным документом: контракт, модель стоимости, инвариант, сложность, память, проверка и ограничения. Начинайте с простейшего представления, которое выполняет контракт, и усложняйте его по установленной причине. Для множества в RAM причиной может быть строгая задержка; для дискового словаря — случайные записи; для карты — число ложных кандидатов.

Структура	Сильная сторона	Существенное ограничение
Skip list	Простой упорядоченный интерфейс	Время ожидаемое, возможен длинный путь
2–3-дерево	Логарифмическая высота в худшем случае	Расщепления, перераспределения, слияния
LSM-tree	Пакетная организация записей	Фоновая работа и усиление чтения/записи
R-семейство	Отбор пространственных кандидатов	Перекрытие и точный геометрический фильтр
DSU с откатом	Запросы по известной истории	Требует офлайн-обработки удалений
Куча Бродала–Окасаки	Константные вставка и слияние	Сложные инварианты, отдельный контракт

Самостоятельная работа. Сравните skip list и `std::set` на одинаковой последовательности операций. Сначала проверьте ответы, затем измерьте время и память. Отдельно рассмотрите случайные, возрастающие и чередующиеся ключи; несколько seed; разное соотношение чтений и изменений. Укажите компилятор, параметры сборки, объём данных и способ измерения. Один удачный замер не доказывает превосходство структуры на всех нагрузках.

Второе упражнение — восстановить работу офлайн-связности на треугольнике из 11.9. Нарисуйте интервалы жизни рёбер, их покрытие деревом времени и журнал отката на одном пути. Третье — определить, какие версии записей LSM можно удалить при наличии старого снимка. Эти упражнения связывают завершающий обзор с деревьями отрезков, амортизацией и персистентностью.

Все одиннадцать модулей плана теперь образуют последовательный текст: от стоимости операций и диапазонных агрегатов до сохранения версий и выбора прикладной структуры. Продвинутые конструкции в обзорных разделах остаются направлениями для семинарского углубления; полный программный разбор этой главы — skip list.

11.12. Практика к главе

Прямых задач на skip list, LSM и пространственные индексы в просмотренных архивах iRunner не найдено. Выполните авторские упражнения этой главы: сравнение skip list со std::set, трассировку DSU с откатом и анализ версий LSM при наличии старого снимка. Эти упражнения не имеют номеров задач iRunner.

11.13. Вопросы для самопроверки

1. Чем отличается ожидаемая оценка от амортизированной на одной фиксированной истории?
2. Почему `lower_bound` перемещается вправо только по ключам, строго меньшим запроса?
3. Выведите ожидаемое число ссылок skip list из распределения высот.
4. Что произойдёт с поиском, если предел высоты оставить равным 3 при росте n?
5. Как расщепление корня 2–3-дерева сохраняет равную глубину листьев?
6. Почему удаление ключа в LSM требует отдельной записи, а не только очистки буфера?
7. Почему логарифмическая высота R-дерева не гарантирует единственный путь запроса?
8. Как треугольник опровергает попытку удалить ребро простым откатом одного union?
9. Почему в DSU с простым откатом обычно не используют сжатие путей?
10. Какая нижняя граница мешает всем операциям сравнивающей очереди быть константными?

(c) Aleksey Tolstikov